

Simulador de ATM Bancário (C++11 & Qt)

Um simulador de Caixa Eletrônico (Automated Teller Machine) robusto desenvolvido com C++11 e Qt Framework. O projeto implementa as operações bancárias essenciais utilizando os princípios da Programação Orientada a Objetos (POO) e apresenta uma interface gráfica moderna baseada no Fluent Design (Windows 11) através de Qt Style Sheets (QSS).

Projeto desenvolvido para fins acadêmicos e de demonstração de arquitetura de software (Máquina de Estados e MVC).

Funcionalidades

Autenticação Segura: Validação de número de conta e PIN com bloqueio lógico após 3 tentativas inválidas.

Saques e Depósitos: Operações transacionais com cálculo atômico de saldo.

Pagamento de Contas: Validação rígida de código de barras (boletos de 47/48 dígitos) utilizando Expressões Regulares (`std::regex`).

Extrato Detalhado: Histórico de movimentações (entradas e saídas) exibido em tabela dinâmica com persistência de dados.

UI/UX Moderna: Interface responsiva e animada gerida por `QStackedWidget`, simulando o design limpo do Windows 11.

Arquitetura Polimórfica: Uso de herança e métodos virtuais puros para gerenciar diferentes tipos de transações.

Tecnologias e Ferramentas

C++ (Padrão ISO C++11)

Framework: Qt 5 / Qt 6 (Core, Gui, Widgets)

IDE: Qt Creator

Estilização: Qt Style Sheets (QSS)

Controle de Versão: Git & GitHub

Como Compilar e Executar

Pré-requisitos

Certifique-se de ter instalado em sua máquina:

Qt Framework (Incluindo o Qt Creator)

Compilador C++ (MinGW no Windows ou GCC/Clang no Linux/macOS)

Passos para Instalação

Clone este repositório:

Bash

```
git clone https://github.com/danieleduardoaluno/CJOPROO2.git
```

Abra o projeto no Qt Creator:

Inicie o Qt Creator.

Vá em File > Open File or Project...

Selecione o arquivo ATM_Project.pro localizado na raiz do repositório clonado.

Configure o Build:

Selecione o Kit de compilação apropriado (ex: Desktop Qt MinGW 64-bit).

Compile e Rode:

Pressione Ctrl + R (ou clique no botão verde de "Play" no canto inferior esquerdo) para compilar e iniciar a aplicação.

Lógica Principal (Máquina de Estados)

O fluxo do sistema é gerenciado por uma arquitetura de estados. A transição entre os estados altera a interface visível (QStackedWidget) e as permissões de ação do usuário:

IDLE -> Aguardando inserção de cartão.

AUTHENTICATING -> Validando credenciais do banco de dados.

MAIN_MENU -> Ocioso aguardando escolha da transação.

PROCESSING -> Calculando saldo e gravando histórico (Smart Pointers em ação).

Como Contribuir

Contribuições são sempre bem-vindas! Para contribuir:

Faça um Fork do projeto.

Crie uma branch para sua feature (git checkout -b feature/MinhaNovaFeature).

Faça o commit das suas alterações (git commit -m 'Adiciona validação XYZ').

Faça o push para a branch (git push origin feature/MinhaNovaFeature).

Abra um Pull Request.

Licença

Este projeto está sob a licença MIT. Sinta-se livre para usá-lo, modificá-lo e distribuí-lo conforme necessário para fins educacionais ou comerciais.

Desenvolvido com C++ por danielduardoaluno